

Содержание:

1. Введение.....	3
1.1. Цель.....	3
1.2. Ссылки.....	3
1.3. Термины и обозначения.....	3
2. Среда верификации.....	5
2.1. Связь с другими процессами ЖЦ ПО.....	5
2.2. Независимость.....	5
2.3. Инструменты верификации.....	5
2.4. Методы верификации.....	6
3. Мероприятия процесса верификации.....	8
3.1. Формальная инспекция.....	8
3.1.1. Входные данные.....	8
3.1.2. Критерии начала мероприятия.....	9
3.1.3 Процедура выполнения мероприятия.....	9
3.1.3.1 Инициализация.....	9
3.1.3.2 Экспертиза и обсуждение.....	9
3.1.3.2 Вынесения решения и регистрация результатов.....	10
3.1.4 Критерии окончания мероприятия.....	10
3.1.5 Выходные данные.....	10
3.2. Разработка тестовых примеров.....	10
3.2.1. Входные данные.....	10
3.2.2. Выходные данные.....	11
3.2.3. Критерии начала мероприятия.....	11
3.2.4. Процедура выполнения мероприятия.....	11
3.2.5 Критерий окончания мероприятия.....	12
3.3. Формальный прогон тестовых примеров.....	12
3.3.1. Входные данные.....	12
3.3.2. Выходные данные.....	12
3.3.3. Критерии начала мероприятия.....	12
3.3.4. Процедура выполнения мероприятия.....	12
3.3.5 Критерий окончания мероприятия.....	13
3.4. Анализ трассировки.....	13
3.4.1. Входные данные.....	13
3.4.2. Выходные данные.....	13
3.4.3. Критерии начала мероприятия.....	13
3.4.4. Процедура выполнения мероприятия.....	14
3.4.5. Критерий окончания мероприятия.....	14
3.5. Анализ структурного покрытия.....	14
3.5.1. Входные данные.....	14

3.5.2. Выходные данные.....	15
3.5.3. Критерии начала мероприятия.....	15
3.5.4. Процедура выполнения мероприятия.....	15
3.5.5. Критерий окончания мероприятия.....	16
4. Дополнительно.....	17
4.1. Стенд отладки испытаний.....	17

1. Введение

1.1. Цель

Данный документ определяет порядок проведения мероприятий верификации программного обеспечения, разработанного в рамках проекта “Система автоматизированного развертывания окружений”

1.2. Ссылки

В данном документе содержатся ссылки на следующие документы:

1. План разработки
2. Стандарт на разработку требований
3. План управления конфигурацией
4. Стандарт на проектирование
5. Стандарт на кодирование

1.3. Термины и обозначения

В данном документе используются следующие термины (понятия):

Понятие	Определение
Развертывание	Все действия, которые делают программную систему готовой к использованию, в том числе запуск исполняемых файлов приложения на удаленном или локальном сервере в изолированном окружении с целью работы с приложением или проведения ручного тестирования приложения
Система	Совокупность инструментов для проведения развертывания приложения и контроля за состоянием этого приложения
Репозиторий	Хранилище кода программы и необходимых для запуска программы файлов
Проект	Объект в системе, представляющий собой совокупность сервисов, запущенных на некотором сервере, а также связанный с кодом этих сервисов, хранящимся в некотором репозитории. В рамках системы является минимальной единицей (сущностью), подлежащей развертыванию

Также используются следующие сокращения:

Сокращение	Обозначение
ЖЦ	Жизненный цикл
ПО	Программное обеспечение
УК	Управление конфигурацией
ФИ	Формальная инспекция

2. Среда верификации

2.1. Связь с другими процессами ЖЦ ПО

Входными данными процесса верификации являются:

- Планы проекта (План разработки, План УК ПО, План верификации)
- Описание проекта
- Требования к ПО
- Исходный код
- Выходы процесса верификации

Выходными данными являются:

- Тестовые примеры и процедуры
- Сообщения о проблемах
- Результаты испытаний (прогона тестовых примеров)
- Протокол ФИ и перечень выявленных несоответствий

2.2. Независимость

Независимость мероприятий процесса верификации ПО заключается в том, что мероприятие верификации выполняется лицом (или лицами), не являющимися разработчиками проверяемого объекта.

2.3. Инструменты верификации

Инструменты, которые будут использованы в процессе верификации ПО, а также их назначение:

Название инструмента	Назначение инструмента
Стенд отладки и испытаний (см. п.4.1)	Проведение испытаний
VS Code	Среда разработки для написания кода тестов
Фреймворк Selenium	Выполнение тестовых примеров на стенде отладки и испытаний для

	проведения испытаний интеграции ПО
Библиотека “testing”	Выполнение тестовых примеров на стенде отладки и испытаний для проведения испытаний низкого уровня (с использованием языка Go)
Postman	Ручное выполнение тестовых примеров на стенде отладки и испытаний для проведения испытаний
Docker	Создание исполняемых пакетов, содержащий все необходимое для запуска верифицируемого ПО: объектный код, среду выполнения, библиотеки, переменные окружения и файлы конфигурации

2.4. Методы верификации

В данном разделе описаны методы, которые планируется использовать для проведения мероприятий процесса верификации ПО.

2.4.1. Рассмотрение

Рассмотрение – это метод верификации, который позволяет получить качественную оценку правильности. В рамках работы над данным проектом все рассмотрения проводятся в форме формальной инспекции (см. раздел 2.4.4)

2.4.2. Анализ

Анализ – это метод верификации, который позволяет получить повторно воспроизводимое свидетельство корректности анализируемых данных ЖЦ ПО.

Если при процессе анализа были выявлены несоответствия, то они должны быть задокументированы в виде сообщений о проблемах и переданы в процесс, результат которого анализировали.

2.4.3. Испытания

Испытания – это метод верификации, при котором сравниваются ожидаемые от ПО выходные данные (которые составлены на основе требований к ПО), и фактически полученные в результате воздействия на ПО тестовым набором данных. При этом, тестовые данные создаются на основании требований к ПО и должны обеспечивать полное покрытие требований.

Целью испытаний является демонстрация того, что ПО удовлетворяет тем требованиям, которые к нему предъявляются. Но при этом важно понимать, что проведение испытаний не может доказать правильное поведение ПО при всех возможных входных данных.

2.4.4. Формальная инспекция

Формальная инспекция – метод оценки соответствия результатов входным данным, на основании которых они получены, заключается в выполнении процедур по формальным правилам (см. раздел 3.1.), которые способствуют обнаружению и регистрации ошибок.

При проведении инспекции используются специальные проверочные перечни, причем для каждого типа объекта инспекции (см. раздел 3.1.) необходим свой перечень. Перечни утверждаются руководителем проекта и хранятся в системе Confluence (см. План управления конфигурацией ПО).

3. Мероприятия процесса верификации

3.1. Формальная инспекция

В данном разделе описан процесс формальной инспекции, применимый для любого типа объекта инспекции.

В контексте мероприятия формальной инспекции выделены следующие роли:

- Автор – один из разработчиков объекта инспекции. Если разработчик недоступен, на роль автора назначается сотрудник, который будет исправлять обнаруженные несоответствия в объекте инспекции.
- Инспектор – сотрудник, хорошо знающий технические подробности проекта, относящиеся к объекту инспекции, и отвечающий за обнаружение несоответствий.
- Ведущий – сотрудник, отвечающий за надлежащее выполнение процесса формальной инспекции, начиная с момента назначения его на эту роль до завершения инспекции.

На роль инспекторов рекомендуется назначать сотрудников, имеющих опыт работы в указанных процессах. Один сотрудник может закрывать несколько пунктов в списке процессов, равно как и допустимо назначать нескольких инспекторов из одного процесса. При назначении Инспекторов должны выполняться требования независимости.

Формальной инспекции подлежат следующие типы объектов инспекции:

- Планы и стандарты проекта,
- Требования к ПО высокого уровня,
- Описание проекта ПО,
- Исходный код,
- Тестовые примеры и процедуры,
- Результаты прогона тестовых примеров.

3.1.1. Входные данные

Входными данными процесса являются:

1. План верификации ПО;
2. Запрос на изменение (см. УК ПО);
3. Объекты формальной инспекции;

4. Входные данные процесса, результаты которого являются объектом формальной инспекции;
5. Запись о предыдущей формальной инспекции (при наличии)

3.1.2. Критерии начала мероприятия

Установлены базовые версии входных данных, на основе которых разработаны объекты формальной инспекции.

3.1.3 Процедура выполнения мероприятия

3.1.3.1 Инициализация

Автор помещает объекты инспекции в GitLab. При этом в записи о формальной инспекции устанавливается связь проводимой формальной инспекции с последними версиями всех единиц конфигурации объектов формальной инспекции. Руководитель проекта назначает Ведущего и Инспекторов для данной формальной инспекции.

3.1.3.2 Экспертиза и обсуждение

Инспекторы проверяют существенные характеристики объектов инспекции, указанных на титульном листе записи о формальной инспекции. Обнаруженные несоответствия должны быть сформулированы и записаны в списке замечаний в записи о формальной инспекции.

Для самоконтроля Инспектора при проведении формальных инспекций применяются заранее подготовленные проверочные перечни, которые утверждаются руководителем проекта и хранятся в системе Confluence (см. План управления конфигурацией ПО). Перечни содержат список вопросов, которые позволяют оценить соответствие объектов формальной инспекции входным данным (хотя инспектор может обнаружить несоответствия, не связанные с вопросами из контрольного перечня)

Если найденное несоответствие таково, что хотя бы одно из положений, описываемых каким-либо вопросом проверочного перечня, не выполняется, то Инспектор должен ответить “Нет” на данный вопрос и связать с ним замечание, описывающее найденное несоответствие. В общем случае, одно замечание может быть связано с несколькими вопросами проверочного перечня, и наоборот, с одним вопросом перечня может быть связано несколько замечаний.

Если выявлено несоответствие во входных данных, не являющихся объектом формальной инспекции, то вместо замечания Инспектор должен открыть сообщение о проблеме, описывающее несоответствие.

3.1.3.2 Вынесения решения и регистрация результатов

Если ФИ не выявила несоответствий, то проверенная версия объекта инспекции становится частью базовой версии. Если ФИ выявила несоответствия, то по перечню несоответствий создаются сообщения об ошибках, затем запросы на изменения, а объект инспекции отправляется на доработку. Ожидается, что в таком случае в будущем будет проведена повторная формальная инспекция данного объекта инспекции.

3.1.4 Критерии окончания мероприятия

Инспектор ответил на все вопросы контрольного перечня, сформулировав все отмеченные несоответствия в виде замечаний и отразив их в записи о формальной инспекции.

3.1.5 Выходные данные

- Сообщения о проблемах;
- Запись о формальной инспекции, состоящая из:
 - Титульного листа формальной инспекции,
 - Списка замечаний,
 - Заполненных проверочных перечней.

3.2. Разработка тестовых примеров

Тестирование ПО используется, чтобы продемонстрировать, что ПО удовлетворяет предъявляемым к нему требованиям, и продемонстрировать с высокой степенью уверенности, что были устранены ошибки, которые могли привести к недопустимым отказным состояниям, как это определено процессом оценки безопасности системы.

3.2.1. Входные данные

Входными данными процесса кодирования тестовых примеров являются:

1. План верификации ПО
2. Запрос на изменение (см. УК ПО)
3. Стандарт на кодирование
4. Требования к ПО
5. Описание проекта

3.2.2. Выходные данные

К выходным данным процесса относятся:

1. Сообщения о проблемах
2. Тестовые примеры

Причем тестовые примеры должны содержать:

- Набор входных воздействий
- Сценарий выполнения
- Тестовые процедуры
- Тестовые заглушки (при необходимости)
- Набор ожидаемых результатов (выходов)
- Критерии оценки результатов выполнения

3.2.3. Критерии начала мероприятия

Критерием начала мероприятия является установка базовых версий всех единиц конфигурации из списка входных данных процесса.

3.2.4. Процедура выполнения мероприятия

В рамках мероприятия по разработке тестовых примеров реализуются тесты низкого уровня и интеграционные тесты.

Тесты низкого уровня реализуются в виде тестовых функций на языке Go с использованием библиотеки “testing” (см. п. 2.3) и с соблюдением стандарта на кодирование (раздел “Написание тестовых процедур”).

Интеграционные тесты реализуются в виде скриптов на языке программирования Python с использованием фреймворка “Selenium” (см. п. 2.3) с соблюдением стандарта на кодирование (раздел “Написание тестовых процедур”). Также интеграционные тесты могут быть реализованы с помощью инструмента “Postman” (см. п. 2.3), в виде скриптов, осуществляющих http запросы к

разрабатываемому ПО. При обнаружении любых несоответствий со входными данными создается сообщение о проблеме.

3.2.5 Критерий окончания мероприятия

Критерием окончания мероприятия является реализация запроса на изменение и предоставление кода реализации, набора входных воздействий, сценария выполнения, тестовых загрузок (при необходимости), набора ожидаемых результатов (выходов) для каждого тестового примера или группы тестовых примеров.

3.3. Формальный прогон тестовых примеров

3.3.1. Входные данные

Входными данными процесса испытания ПО являются:

1. План верификации ПО
2. Исполняемый объектный код
3. Тестовые примеры

3.3.2. Выходные данные

К выходным данным процесса относятся:

1. Сообщения о проблемах
2. Результаты прогона тестовых примеров

3.3.3. Критерии начала мероприятия

Критерием начала мероприятия является установка базовых версий всех единиц конфигурации из списка входных данных процесса.

3.3.4. Процедура выполнения мероприятия

1. Тест-инженер извлекает тестовые примеры из GitLab и проводит формальный прогон тестовых примеров с использованием соответствующей среды испытаний.

2. Результаты формального прогона фиксируются в GitLab.

3. Каждая версия результатов прогона трассируется на версии тестовых примеров и соответствующую версию исходного кода, трассировка осуществляется

через создание текстового файла с идентификаторами теста, исполняемого кода и результатов прогона, который затем сохраняется в GitLab вместе с результатами прогона.

4. При обнаружении несоответствий при проведении формального прогона тест-инженер открывает сообщение о проблеме с описанием несоответствий.

3.3.5 Критерий окончания мероприятия

Критерием окончания мероприятия является выполнение следующих условий:

- Тестовые примеры прогнаны с использованием соответствующей среды испытаний;
- Найденные несоответствия (при наличии) описаны через механизм сообщения о проблеме;
- Результаты прогона тестовых примеров сохранены в GitLab.

3.4. Анализ трассировки

3.4.1. Входные данные

Ко входным данным этого мероприятия относится:

1. Требования к ПО
2. План верификации ПО
3. Описание проекта ПО
4. Исходный код
5. Тестовые примеры
6. Каталог комплектации

3.4.2. Выходные данные

1. Протокол анализа трассировки
2. Сообщения о проблемах

3.4.3. Критерии начала мероприятия

Базовые версии требований к ПО, описания проекта ПО, исходного кода, а также тестовых примеров установлены и включены в Каталог комплектации.

3.4.4. Процедура выполнения мероприятия

1. Средствами GitLab и Confluence (см. План разработки) менеджер изменений формирует отчет о трассировке для текущего Каталога комплектации
2. Данные из отчета о трассировке проверяются на предмет того, что для всех входных трассируемых данных присутствуют выходные трассируемые данные:

Трассируемые данные (входные)	Трассируемые данные (выходные)
Требования к системе	Требования к ПО высокого уровня
Требования к ПО высокого уровня	Требования к ПО низкого уровня
Требования к ПО низкого уровня	Модуль исходного кода
Требования к ПО высокого уровня и низкого уровня	Процедура испытаний ПО

3. В случае обнаружения несоответствий необходимо открыть сообщения о проблемах

3.4.5. Критерий окончания мероприятия

Отчет о трассировке сформирован и сохранен в GitLab, открыты сообщения о проблемах для всех найденных несоответствий

3.5. Анализ структурного покрытия

3.5.1. Входные данные

К входным данные мероприятия относятся:

1. Требования к ПО
2. Описание проекта ПО
3. План верификации
4. Исходный код
5. Тестовые примеры

3.5.2. Выходные данные

1. Сообщения о проблемах
2. Отчет об анализе структурного покрытия

3.5.3. Критерии начала мероприятия

Критерием начала данного мероприятия является выполнение прогона всех тестовых примеров в соответствующей среде испытаний.

3.5.4. Процедура выполнения мероприятия

1. Тест-инженер использует тестовые примеры и осуществляет прогон данных примеров в среде испытаний: на компьютере с установленными библиотеками “testing” и “Selenium” (см. п.2.3), а также в режиме “Сбор покрытия”, и, таким образом, получает данные о структурном покрытии.
2. На основе полученных данных автоматически формируется отчет о структурном покрытии. В данном случае необходимо создать отчет о покрытии уровня “Покрытие решений” (“Decision Coverage”)
3. Тест-инженер сравнивает данные, полученные в результате прогона тестовых примеров в инструментированной среде испытания, с данными, полученными в результате формального прогона. Если данные не совпадают, то использование полученного отчета о покрытии не допускается, и тест-инженер пробует повторить прогоны теста, возможно с изменением настроек инструментов прогона. Если невозможно добиться совпадения результатов прогона тестовых примеров при формальном прогоне и при сборе покрытия, то создается сообщение о проблеме.
4. Инженер использует сформированный ранее отчет для проведения анализа структурного покрытия. При этом, сообщение о проблеме создаются в следующих случаях:
 - a. Неполнота тестовых примеров. Создается сообщение о проблеме на доработку тестовых примеров, либо создании новых.
 - b. Неполнота требований. Создается сообщение о проблеме либо о доработке требований (разработке новых), либо об удалении кода, для которого не существует требований.

- с. Мертвый код. Создается сообщение о проблеме для решения об удалении неиспользуемого кода

Также, стоит отметить, что во время выполнения анализа инженер должен заполнять отчет об анализе структурного покрытия, в котором указаны версии требований к ПО, описания проекта ПО, кода и тестов, используемых для получения данных о покрытии, а также перечислены случаи непокрытия и созданные в соответствии с этим сообщения о проблемах.

- 5. Отчет об анализе структурного покрытия сохраняется в GitLab.

3.5.5. Критерий окончания мероприятия

Критерием окончания мероприятия является выполнение следующих условий:

1. Несоответствия (если такие были найдены) зафиксированы в виде сообщений о проблемах.
2. Был сформирован и сохранен отчет об анализе структурного покрытия.

4. Дополнительно

4.1. Стенд отладки испытаний

Стенд отладки испытаний в данном случае представляет собой сервер с запущенным на нем разрабатываемом ПО, исполняемые файлы которого собраны из базовой версии исходного кода разрабатываемого ПО.